

BACHELOR THESIS
Vorname Nachname

Providerblinder, zensurresistenter Netzwerk-Tunnel: Entwurf, Implementierung und Parameterstudie

FAKULTÄT INFORMATIK UND DIGITALE GESELLSCHAFT

Faculty of Computer Science and Digital Society

Vorname Nachname

Providerblinder, zensurresistenter Netzwerk-Tunnel: Entwurf, Implementierung und Parameterstudie

Bachelorarbeit eingereicht im Rahmen der Bachelorprüfung
im Studiengang *Bachelor of Science Angewandte Informatik*
der Fakultät Informatik und Digitale Gesellschaft
der Hochschule für Angewandte Wissenschaften Hamburg

Betreuender Prüfer: Prof. Dr. Vorname Nachname
Zweitgutachter: Prof. Dr. Vorname Nachname

Eingereicht am: TT. Monat 2026

Vorname Nachname

Thema der Arbeit

Providerblinder, zensurresistenter Netzwerk-Tunnel:
Entwurf, Implementierung und Parameterstudie

Stichwörter

Zero-Knowledge, providerblind, QUIC, Noise, Proof-of-Work, Zensurresistenz, NAT-Traversal

Kurzzusammenfassung

Diese Arbeit entwirft, implementiert und evaluiert einen providerblinden, zensurresistenten Netzwerk-Tunnel. Der Betreiber der Vermittlungsebene relayt ausschliesslich Chifretext (Ende-zu-Ende-Verschlüsselung mit dem Noise-Protokoll ueber QUIC), kennt weder Ziel-Hostnamen noch Nutzlast und ist damit strukturell blind gegenueber dem vermittelten Verkehr. Ein Proof-of-Work-gestuetztes Rendezvous begrenzt Missbrauch, ein direkter Peer-to-Peer-Pfad umgeht bei Erreichbarkeit den Relay, und ein TLS-ueber-TCP-Fallback traegt denselben Tunnel, wenn UDP blockiert ist. Eine Parameterstudie am fertigen Produkt vermisst die Roundtrip-Latenz unter emulierten Netzbedingungen ueber die Betriebsarten One-shot, Streaming und Datagramm.

Vorname Nachname

Title of Thesis

Provider-blind, Censorship-resistant Network Tunnel: Design, Implementation and Parameter Study

Keywords

zero-knowledge, provider-blind, QUIC, Noise, proof-of-work, censorship resistance, NAT traversal

Abstract

This thesis designs, implements and evaluates a provider-blind, censorship-resistant network tunnel. The operator of the mesh plane relays only ciphertext (end-to-end encryption with the Noise protocol over QUIC), knows neither the destination hostname nor the payload and is thus structurally blind to the traffic it forwards. A proof-of-work rendezvous limits abuse, a direct peer-to-peer path bypasses the relay when reachable, and a TLS-over-TCP fallback carries the same tunnel when UDP is blocked. A parameter study on the finished product measures round-trip latency under emulated network conditions across the one-shot, streaming and datagram operating modes.

Inhaltsverzeichnis

Abbildungsverzeichnis	xi
Tabellenverzeichnis	xiii
Abkürzungen	xv
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	1
1.3 Zielsetzung und Forschungsfragen	2
1.4 Beitrag	3
1.5 Aufbau der Arbeit	3
2 Grundlagen	4
2.1 Providerblindheit und das Zero-Knowledge-Prinzip	4
2.2 QUIC als Transport	5
2.3 Ende-zu-Ende-Kryptographie mit Noise	5
2.4 Proof-of-Work als Missbrauchsschranke	6
2.5 NAT-Traversal und direkter Pfad	7
2.5.1 NAT-Typen und ihre Bedeutung fuers Hole Punching	7
3 Verwandte Arbeiten	9
3.1 Verschlüsselte Tunnel und VPNs	9
3.2 Anonymitätsnetze	9
3.3 Providerblinde Vermittlung	10
3.4 Proxying ueber HTTP und QUIC	10
3.5 Zensurumgehung	10
3.6 Einordnung	11

4	Anforderungen und Bedrohungsmodell	12
4.1	Funktionale Anforderungen	12
4.2	Nicht-funktionale Anforderungen	13
4.3	Bedrohungsmodell	13
4.3.1	Akteure und Faehigkeiten	13
4.3.2	Vertrauensgrenzen und Annahmen	14
4.4	Schutzziele und Nicht-Ziele	14
5	Architektur	15
5.1	Komponenten und Topologie	15
5.2	Schluesselfluesse	15
5.3	Rollen-Dispatch auf einem Stream	17
5.4	Entwurfsentscheidungen	18
6	Implementierung	19
6.1	Workspace-Struktur	19
6.2	Gemeinsame Bausteine (ct-common)	19
6.3	Rollen-Dispatch am Edge (ct-edge)	21
6.4	Daten- und Steuerpfad (agent, client, control-plane)	21
6.5	Beobachtbarkeit	22
6.6	Guthaben-gedechte Token-Ausgabe	22
7	Produktivierung	23
7.1	Durabler Zustand	23
7.2	Identitaet und Authentifizierung	23
7.3	PKI und Transportsicherheit	24
7.4	Auslieferung	24
7.5	Haertung	24
7.6	Bezahlung	25
7.7	Zusammenfassung	25
8	Evaluation	26
8.1	Testbett und Methodik	26
8.2	Ergebnisse	26
8.3	Diskussion und Limitierungen	27
8.4	Sicherheitsbewertung der Produktivierung	28

9 Diskussion	31
9.1 Beantwortung der Forschungsfragen	31
9.2 Schutzziele gegen das Bedrohungsmodell	32
9.3 Offene Risiken und Grenzen	32
9.4 Methodische Einordnung	32
10 Fazit und Ausblick	34
10.1 Zusammenfassung	34
10.2 Ausblick	35
Literaturverzeichnis	36
A Reproduzierbarkeit	38
A.1 Build und Tests	38
A.2 Testbett-Smokes	38
A.3 Messung und Auswertung	39
A.4 Bau dieser Arbeit	39
Glossar	40
Selbstständigkeitserklärung	41

Abbildungsverzeichnis

5.1	Topologie: Der Edge relayt zwischen Client und Agent, sieht aber nur den Ende-zu-Ende mit Noise verschluesselten Verkehr (gestrichelter Bogen). Die Steuerebene koordiniert (gepunktet), traegt aber keine Nutzlast. . . .	16
8.1	Tunnel-Roundtrip gegen Edge-Verzoegerung (Modus <code>single</code>), je eine Kurve pro Verlust; obere Fehlerbalken bis p95.	28
8.2	Vergleich der Betriebsarten (Verlust 0 %): <code>single</code> , <code>stream</code> und <code>udp</code> ueberlagern sich nahezu.	29
8.3	Tunnel-Roundtrip gegen Paketverlust (Modus <code>single</code>), je eine Kurve pro Verzoegerung.	30

Tabellenverzeichnis

3.1	Verwandte Systeme entlang der zentralen Eigenschaften. ● = erfuehlt, ○ = teilweise, – = nicht erfuehlt.	11
5.1	Wire-Format des Rollen-Dispatch (vereinfacht); alle Tokens sind opak, bezeichnet Konkatination.	17
6.1	Crates des Workspace und ihre Verantwortung.	20
8.1	Tunnel-Roundtrip-Latenz je Betriebsart unter emulierten Netzbedingungen (tc netem); Mittel mit 95%-Konfidenzintervall.	27
8.2	Angreifer, wirksame Kontrolle und Restrisiko.	29

Abkürzungen

CI Konfidenzintervall.

E2E Ende-zu-Ende.

ICE Interactive Connectivity Establishment.

NAT Network Address Translation.

P2P Peer-to-Peer.

PoW Proof-of-Work.

PTO Probe Timeout.

QUIC Quick UDP Internet Connections (RFC 9000).

TCP Transmission Control Protocol.

TLS Transport Layer Security.

UDP User Datagram Protocol.

VPN Virtual Private Network.

1 Einleitung

1.1 Motivation

Wer heute einen Netzwerkdienst durch fremde Infrastruktur tunnelt – sei es ein Virtual Private Network (VPN)-Anbieter, ein Reverse-Proxy oder ein Relay-Netz –, vertraut dem Betreiber dieser Infrastruktur weit mehr an, als technisch notwendig waere. In den ueblichen Architekturen sieht der Vermittler mindestens die Metadaten der Verbindung: welcher Client zu welchem Ziel-Hostnamen spricht, oft auch, wie viel und wann. Bei TLS-terminierenden Proxies liegt zeitweise sogar der Klartext vor. Dieses Wissen ist ein Angriffspunkt: es kann protokolliert, verkauft, unter Zwang herausgegeben oder zur Zensur einzelner Ziele genutzt werden. Anonymisierungsnetze wie Tor [2] adressieren einen Teil des Problems, verlagern das Vertrauen aber auf die Zusammensetzung des Relay-Netzes und sind gegen aktive Netzsperrern (etwa das Blockieren bekannter Einstiegspunkte oder ganzer Transportprotokolle) nur begrenzt robust.

Die Gegenthese dieser Arbeit lautet: Ein Vermittler soll das, was er nicht wissen muss, auch strukturell nicht wissen *koennen*. Ein solcher Dienst ist providerblind – er relayt ausschliesslich Chiffretext, kennt weder Ziel-Hostnamen noch Nutzlast und kann folglich weder das eine noch das andere preisgeben. Vertrauen wird so nicht organisatorisch (durch Richtlinien und Wohlverhalten des Betreibers) hergestellt, sondern kryptographisch erzwungen.

1.2 Problemstellung

Ein providerblinder Tunnel muss mehrere, teils gegenlaeufige Anforderungen gleichzeitig erfuellen:

- **Ende-zu-Ende-Vertraulichkeit gegen den Betreiber.** Die Nutzlast muss zwischen Client und Ziel (*Origin*) verschlüsselt sein, sodass die Vermittlungsebene nur opake Frames weiterreicht. Das schliesst eine TLS-Terminierung am Relay aus und verlangt eine eigene Ende-zu-Ende (E2E)-Schicht – hier das Noise-Protokoll [8] ueber Quick UDP Internet Connections (RFC 9000) (QUIC) [6, 11].
- **Adressierung ohne Hostnamen.** Der Client muss den richtigen Endpunkt erreichen, ohne dem Betreiber einen Hostnamen zu nennen. Das verlangt eine Adressierung ueber opake Tokens und ein Rendezvous, das nicht auf Klarnamen beruht.
- **Missbrauchsresistenz ohne Identitaeten.** Gerade weil der Dienst blind ist, kann er Missbrauch nicht am Inhalt erkennen. Ein Proof-of-Work (PoW)-Nachweis [1] verteuert die Verbindungsanbahnung und begrenzt so Flutangriffe, ohne Identitaeten zu fordern.
- **Zensur- und Network Address Translation (NAT)-Realitaeten.** Reale Netze blockieren zuweilen User Datagram Protocol (UDP) (und damit QUIC) oder liegen hinter NAT. Der Tunnel braucht daher einen Transport-Fallback und einen direkten Pfad, der den Relay bei Erreichbarkeit umgeht [5].

Der Kern der Spannung liegt darin, diese Eigenschaften *zusammen* zu realisieren: Blindheit erschwert Missbrauchsabwehr, Ende-zu-Ende-Kryptographie erschwert Vermittlung, und Zensurresistenz erschwert die Transportwahl.

1.3 Zielsetzung und Forschungsfragen

Ziel der Arbeit ist der Entwurf, die prototypische Implementierung und die empirische Evaluation eines providerblinden Tunnels, der die genannten Anforderungen in einem lauffaehigen System vereint. Daraus leiten sich drei Forschungsfragen ab:

- FF1 (Realisierbarkeit).** Laesst sich ein providerblinder Tunnel so konstruieren, dass der Betreiber nachweislich nur Chiffretext sieht, und zugleich Rendezvous, PoW-Gating, direkter Pfad und Transport-Fallback funktionieren?
- FF2 (Latenz-Overhead).** Welchen Roundtrip-Overhead verursacht der Tunnel gegenueber der reinen Netzlaufzeit, und wie skaliert er mit Verzoegerung und Paketverlust?

FF3 (Netz- und Betriebsartverhalten). Wie verhalten sich die Betriebsarten One-shot, Streaming und Datagramm unter identischen Netzbedingungen, und welche Rolle spielt die PoW-Schwierigkeit?

1.4 Beitrag

Die Arbeit liefert (i) einen in Rust implementierten Prototyp aus fuenf Crates (gemeinsame Wire-Typen, Agent, Edge, Control-Plane, Client), der den vollen Pfad Client $\rightarrow$ Edge $\rightarrow$ Agent $\rightarrow$ Origin mit E2E-Noise, PoW-Rendezvous, direktem Peer-to-Peer (P2P)-Pfad und Transport Layer Security (TLS)-ueber-TCP-Fallback traegt; (ii) ein rein containerbasiertes Testbett, das die Topologie samt Stoergroessen (`tc netem`) emuliert; und (iii) eine Parameterstudie am fertigen Produkt, die den Latenz-Overhead ueber die drei Betriebsarten statistisch belastbar vermisst.

1.5 Aufbau der Arbeit

Kapitel 1 umreisst Motivation und Fragestellung. Die weiteren Kapitel behandeln nacheinander die technischen Grundlagen (Zero-Knowledge-Prinzip, QUIC, Noise, PoW, NAT), verwandte Arbeiten, Anforderungen und Bedrohungsmodell, die Architektur, die Implementierung, die Evaluation mit der Parameterstudie sowie Diskussion und Ausblick.

2 Grundlagen

Dieses Kapitel fuehrt die Konzepte ein, auf denen der Entwurf aufbaut: das Prinzip der Providerblindheit (Abschnitt 2.1), den Transport QUIC (2.2), die Ende-zu-Ende-Kryptographie mit dem Noise-Protokoll (2.3), Proof-of-Work als Missbrauchsschranke (2.4) und die Traversierung von NAT (2.5).

2.1 Providerblindheit und das Zero-Knowledge-Prinzip

Ein Vermittlungsdienst heisst in dieser Arbeit providerblind, wenn er den vermittelten Verkehr strukturell nicht einsehen kann. Der Begriff *Zero-Knowledge* wird hier im woertlichen, systemtechnischen Sinne gebraucht – der Betreiber besitzt „null Wissen“ ueber Ziel und Nutzlast – und nicht im kryptographischen Sinne eines Zero-Knowledge-Beweises. Die Blindheit ist eine *strukturelle* Eigenschaft: sie ergibt sich daraus, dass die vertraulichen Daten den Vermittler nur als Chiffretext erreichen und die Adressierung ueber opake Tokens statt Klarnamen erfolgt. Damit unterscheidet sie sich von einer bloss *organisatorischen* Zusicherung (etwa einer No-Log-Richtlinie), die auf dem Wohlverhalten des Betreibers beruht.

Das Vertrauensmodell verschiebt sich entsprechend: Der Client muss dem Betreiber weder Vertraulichkeit noch Nichtprotokollierung glauben, weil dieser die relevanten Informationen gar nicht erst erhaelt. Er muss ihm lediglich *Verfuegbarkeit* zutrauen – ein Relay kann den Dienst verweigern oder verzoegern, aber nicht mitlesen. Anonymisierungsnetze wie Tor [2] verfolgen ein verwandtes Ziel ueber mehrere Zwischenstationen; der hier gewaehlte Ansatz konzentriert sich stattdessen auf die Blindheit eines einzelnen Vermittlers plus einen direkten Pfad, der diesen bei Erreichbarkeit ganz umgeht.

2.2 QUIC als Transport

QUIC [6] ist ein verbindungsorientiertes Transportprotokoll oberhalb von UDP. Es integriert die Verschlüsselung fest in den Transport: der Handshake nutzt TLS 1.3 [9, 11], so dass bereits die Transportschicht authentisiert und verschlüsselt ist. Gegenüber Transmission Control Protocol (TCP)+TLS bietet QUIC drei für diese Arbeit relevante Eigenschaften: (i) *gemultiplexte Streams* innerhalb einer Verbindung ohne Head-of-Line-Blocking zwischen ihnen, (ii) einen *schnellen Verbindungsaufbau* (1-RTT, mit Sitzungswiederaufnahme 0-RTT) und (iii) Verbindungs-IDs, die einen Pfadwechsel (Connection Migration) erlauben.

Die Stream-Multiplexierung trägt im Entwurf den Rollen-Dispatch: Client, Agent und Steuerbefehle teilen sich eine Verbindung, ohne sich gegenseitig zu blockieren. Weil QUIC auf UDP aufsetzt, ist es zugleich der Punkt, an dem Zensur ansetzen kann – wird UDP blockiert, ist QUIC nicht nutzbar. Der Entwurf braucht daher einen Transport-Fallback (Abschnitt 2.5 und Kapitel 5), der denselben Tunnel über TLS-über-TCP trägt.

Handshake und Datenphase. Der Verbindungsaufbau läuft in QUIC über *Initial*- und *Handshake*-Pakete, in die der TLS-1.3-Handshake eingebettet ist; nach einer Runde (1-RTT) sind die Schlüssel etabliert und die Datenphase (*1-RTT*-Pakete) beginnt. Bei einer zuvor bekannten Gegenstelle erlaubt die Sitzungswiederaufnahme frühe Daten (0-RTT), die jedoch gegen Wiedereinspielung (Replay) abzusichern sind und daher nur für idempotente Anfragen taugen. Streams sind entweder bidirektional oder unidirektional und werden unabhängig flusskontrolliert; Verlust auf einem Stream blockiert die anderen nicht (kein stream-übergreifendes Head-of-Line-Blocking). Erst dies macht den Rollen-Dispatch (Kapitel 5) auf einer einzigen Verbindung effizient: Registrierung, Relay und Steuerbefehle konkurrieren nicht um dieselbe Sequenz.

2.3 Ende-zu-Ende-Kryptographie mit Noise

Damit der Vermittler blind bleibt, genügt die Transportverschlüsselung von QUIC nicht: sie endet am Edge. Die Nutzlast muss zusätzlich zwischen Client und Origin verschlüsselt sein. Dafür dient das Noise Protocol Framework [8], ein Baukasten für authentifizierte Schlüsselaustausch-Handshakes aus Diffie-Hellman-Operationen, einem AEAD-Chiffre und einer Hashfunktion.

Konkret wird das Muster `Noise_IK_25519_ChaChaPoly_BLAKE2s` eingesetzt: Schlüsselaustausch ueber Curve25519 (X25519), Verschlüsselung mit ChaCha20-Poly1305, Hashing mit BLAKE2s. Im Muster `IK` uebertraegt der *Initiator* (Client) seinen statischen Schlüssel waehrend des Handshakes („I“), waehrend der statische Schlüssel des *Responders* (Origin) dem Initiator vorab bekannt ist („K“). Genau diese Vorabkenntnis ist der Hebel fuer die Providerblindheit: der Client *pinnt* die Origin-Identitaet (den oeffentlichen Origin-Schlüssel), die er ausserhalb des Bandes als Teil einer Capability erhaelt. Ein Vermittler, der die Origin-Identitaet nicht kennt, kann den Handshake nicht als Man-in-the-Middle unterlaufen; ein Angreifer mit falschem Schlüssel scheitert am Pinning. Nach dem Handshake gehen beide Seiten in den Transport-Modus ueber und tauschen laengen-praefigierte, verschlüsselte Frames aus, die der Edge nur noch byteweise weiterreicht.

Ablauf des IK-Handshakes. Der Handshake besteht aus zwei Nachrichten. In der ersten sendet der Initiator seinen ephemeren oeffentlichen Schlüssel und – bereits verschlüsselt – seinen statischen Schlüssel; die Diffie-Hellman-Operationen `es` (ephemeral-static) und `ss` (static-static) mischen dabei sowohl den ephemeren als auch den vorab bekannten Origin-Schlüssel in den Sitzungszustand, sodass nur der echte Origin die Nachricht entschlüsseln kann. In der zweiten Nachricht antwortet der Responder mit seinem ephemeren Schlüssel und den Operationen `ee` und `se`; damit ist der Sitzungsschlüssel beidseitig authentisiert und – dank der ephemeren Anteile – *vorwaertsgeheim*: die spaetere Kompromittierung eines statischen Schlüssels legt vergangene Sitzungen nicht offen. Anschliessend leiten beide Seiten je Richtung einen symmetrischen Transportschlüssel ab. Weil der Origin-Schlüssel bereits in die erste Nachricht eingeht, ist der Handshake an die gepinnte Identitaet gebunden, noch bevor Nutzdaten fliessen.

2.4 Proof-of-Work als Missbrauchsschranke

Ein blinder Dienst kann Missbrauch nicht am Inhalt erkennen. Um die Verbindungsanbahnung dennoch zu verteuern, wird ein Proof-of-Work nach dem Hashcash-Prinzip [1] vorgeschaltet. Der Edge stellt eine Challenge aus einem Nonce und einer Schwierigkeit; der Client muss eine Loesung finden, deren Hash eine geforderte Anzahl fuehrender Nullbits aufweist. Das Finden ist – exponentiell in der Schwierigkeit – teuer, die Pruefung dagegen ein einzelner Hash. Dieser asymmetrische Aufwand begrenzt Flut- und

Rendezvous-Missbrauch, ohne dass der Dienst Identitäten führen oder Inhalte sehen müsste. Die Schwierigkeit ist ein Stellparameter, dessen Einfluss die Parameterstudie (Kapitel 8) untersucht.

Kosten und Asymmetrie. Modelliert man den Hash als Zufallsorakel, so trifft ein einzelner Versuch die geforderten d führenden Nullbits mit Wahrscheinlichkeit 2^{-d} ; die erwartete Zahl der Versuche bis zur Lösung ist damit 2^d , während die Prüfung genau einen Hash kostet. Der Aufwand des Clients wächst also exponentiell in der Schwierigkeit d , der des Edge bleibt konstant. Genau diese Asymmetrie macht den Nachweis als Schranke brauchbar: Eine einzelne Anbahnung bleibt für einen ehrlichen Client günstig, während das massenhafte Anbahnen eines Fluters quadratisch bis exponentiell teurer wird. Die Wahl von d ist ein Kompromiss zwischen wirksamer Dämpfung und der Belastung schwacher Clients (Kapitel 9).

2.5 NAT-Traversal und direkter Pfad

Reale Endpunkte liegen häufig hinter NAT, das ausgehende Verbindungen zulässt, eingehende aber ohne Zustand verwirft. Der Relay-Pfad umgeht dies, weil sowohl Client als auch Agent *ausgehend* zum Edge verbinden. Ein *direkter* P2P-Pfad, der den Edge umgeht, muss dagegen die NAT-Bindung überwinden. Das etablierte Verfahren ist das gleichzeitige Öffnen (Hole Punching) auf Basis reflexiver, vom Vermittler beobachteter Adressen [5]; standardisiert ist die Kandidaten-Aushandlung als Interactive Connectivity Establishment (ICE) [7]. Im Entwurf inseriert der Agent einen direkten Listener beim Edge, den der Client entdeckt und bevorzugt nutzt; scheitert der direkte Aufbau, fällt der Client transparent auf den Relay zurück. Echtes Hole Punching über symmetrisches NAT bleibt eine Grenze des flachen Container-Testbetts und wird in Kapitel 8 eingeordnet.

2.5.1 NAT-Typen und ihre Bedeutung für Hole Punching

NAT-Geräte unterscheiden sich darin, wie sie eine einmal geöffnete ausgehende Bindung für eingehenden Verkehr wiederverwenden. Bei einem *Full-Cone*-NAT gilt die Bindung für beliebige externe Absender; bei *Address-Restricted*- bzw. *Port-Restricted*-NAT nur für zuvor kontaktierte Adressen bzw. Adress-Port-Paare. In diesen drei Fällen genügt

das gleichzeitige Oeffnen: beide Seiten senden zuerst hinaus und erzeugen so die noetige Bindung fuer die Gegenrichtung, woraufhin die vom Vermittler beobachtete reflexive Adresse den Peer erreicht. Ein *symmetrisches* NAT dagegen vergibt je Ziel eine *andere* externe Portnummer; die beobachtete reflexive Adresse sagt dann nichts ueber den Port voraus, den der Peer tatsaechlich sehen wird, und das Hole Punching schlaegt ohne zusaetzliche Portvorhersage fehl. Der vorliegende Prototyp setzt auf reflexive Adressen und deckt damit die nicht-symmetrischen Faelle ab; symmetrische NATs bleiben eine bekannte Grenze (Kapitel 9).

3 Verwandte Arbeiten

Der Entwurf beruehrt drei Forschungsstraenge: verschluesselte Tunnel und Anonymitaetsnetze, providerblinde Vermittlung sowie Zensurumgehung. Dieses Kapitel ordnet die vorliegende Arbeit in jeden dieser Straenge ein und arbeitet heraus, worin die Kombination neu ist.

3.1 Verschlüsselte Tunnel und VPNs

Klassische VPN-Loesungen bauen einen verschluesselten Tunnel zwischen Client und einem Konzentrator auf. WireGuard [3] ist hierfuer ein modernes, schlankes Beispiel: es nutzt – wie die vorliegende Arbeit – Bausteine aus dem Noise-Umfeld (Curve25519, ChaCha20-Poly1305, BLAKE2s) und erreicht einen kompakten, gut analysierbaren Handshake. Der entscheidende Unterschied liegt im Vertrauensmodell: der VPN-Server ist der Endpunkt des Tunnels und sieht damit die Ziele des Clients im Klartext. Die Vertraulichkeit gegenueber dem Betreiber ist keine strukturelle Eigenschaft, sondern beruht auf dessen Wohlverhalten. Die vorliegende Arbeit uebernimmt die kryptographische Substanz, verschiebt den Terminierungspunkt der Nutzlast-Verschlüsselung aber vom Vermittler zum Origin.

3.2 Anonymitaetsnetze

Tor [2] entkoppelt Absender und Ziel ueber eine Kette von mindestens drei Relays, von denen keines beide Enden zugleich kennt. Das Schutzziel ist primaer die *Unverkettbarkeit* von Nutzer und Ziel gegenueber einzelnen Relays, erkaufte durch mehrere Zwischenspruenge und entsprechende Latenz. Die vorliegende Arbeit verfolgt ein engeres, aber schaeferes Ziel: die strukturelle Blindheit eines *einzelnen* Vermittlers gegenueber *Inhalt und Ziel*, ergaenzt um einen direkten Pfad, der diesen Vermittler bei Erreichbarkeit

ganz umgeht. Anonymitaet im Tor’schen Sinne ist ausdruecklich kein Ziel; wohl aber die Nicht-Einsehbarkeit durch den Betreiber.

3.3 Providerblinde Vermittlung

Am naechsten steht die Arbeit dem Prinzip von Oblivious HTTP [12]: dort trennt ein Relay das *Wer* vom *Was*, indem es Anfragen an ein Gateway weiterreicht, ohne deren Inhalt zu sehen, waehrend das Gateway den Absender nicht kennt. Oblivious HTTP ist jedoch auf einzelne, kapselbare Anfrage-Antwort-Paare zugeschnitten und setzt ein kooperierendes Gateway voraus. Die vorliegende Arbeit uebertraegt den Grundgedanken – der Vermittler sieht nur Chiffretext – auf einen *allgemeinen, bidirektionalen Tunnel* beliebiger TCP- und UDP-Protokolle, mit Origin-Pinning statt Gateway und mit einem Proof-of-Work-Rendezvous als Missbrauchsschranke.

3.4 Proxying ueber HTTP und QUIC

MASQUE („Proxying UDP in HTTP“) [10] tunnelt UDP ueber HTTP/3 und damit ueber QUIC. Technisch aehneln das dem hier genutzten Transport, das Vertrauensmodell unterscheidet sich jedoch erneut: der MASQUE-Proxy kennt das Ziel (Adresse und Port), weil er selbst die UDP-Assoziation aufbaut. Er ist ein transparenter Weiterleiter, kein blinder Relay. Der Fallback der vorliegenden Arbeit (TLS-ueber-TCP) verfolgt ein verwandtes Ziel wie MASQUE – Betrieb, wenn UDP blockiert ist –, transportiert dabei aber weiterhin nur den bereits Ende-zu-Ende verschluesselten Tunnel.

3.5 Zensurumgehung

Zur Umgehung aktiver Netzsperrn existiert ein breites Feld. Domain Fronting [4] verbirgt das wahre Ziel hinter dem Namen eines grossen, schwer blockierbaren Dienstes; Decoy Routing bzw. Refraction Networking [13] verlagert die Umlenkung in die Netzinfrastruktur selbst. Diese Ansaetze zielen primaer auf die *Unauffaelligkeit* des Datenstroms (Verschleierung gegen einen beobachtenden Zensor). Die vorliegende Arbeit setzt komplementaer an: nicht Verschleierung, sondern strukturelle Blindheit des Vermittlers plus

Tabelle 3.1: Verwandte Systeme entlang der zentralen Eigenschaften. ● = erfuehlt, ○ = teilweise, – = nicht erfuehlt.

System	E2E, Vermittler-blind	Allg. TCP/UDP	QUIC	P2P-Pfad	Missbrauchsschranke
WireGuard [3]	○	●	–	●	–
Tor [2]	○	○	–	–	–
Oblivious HTTP [12]	●	–	–	–	–
MASQUE [10]	–	○	●	–	–
Diese Arbeit	●	●	●	●	●

ein Transport-Fallback, der die haeufigste grobe Sperre – das Blockieren von UDP – ueberbrueckt. Traffic-Analyse-Resistenz gegen einen aktiven Beobachter ist kein Ziel und wird in Kapitel 8 als offene Grenze benannt.

3.6 Einordnung

Die genannten Systeme decken jeweils einen Teil der Anforderungen ab: WireGuard die kryptographische Substanz, Oblivious HTTP die Blindheit, MASQUE den QUIC-Transport, Tor die Entkopplung, die Zensurumgehungs-Systeme die Sperrresistenz. Tabelle 3.1 stellt die Systeme entlang der zentralen Eigenschaften gegenueber und macht sichtbar, dass keines sie vollstaendig vereint.

Die Zeilen bestaetigen die Prosa: WireGuard verschluesselt zwar Ende-zu-Ende, doch jeder Peer kennt seinen Gegenpart (○); Tor entkoppelt zwar Absender und Ziel, ist aber auf TCP zugeschnitten (○) und ohne Direktpfad; Oblivious HTTP ist blind, aber auf Anfrage-Antwort-Paare begrenzt; MASQUE bringt den QUIC-Transport, sein Proxy ist jedoch sehend. Der Beitrag der vorliegenden Arbeit ist die *Kombination* dieser Eigenschaften in einem einzigen, lauffaehigen System: ein providerblinder, allgemein einsetzbarer Tunnel mit Ende-zu-Ende-Pinning, Proof-of-Work-Rendezvous, direktem Peer-to-Peer-Pfad und Transport-Fallback – inklusive empirischer Vermessung des dadurch entstehenden Latenz-Overheads.

4 Anforderungen und Bedrohungsmodell

Dieses Kapitel praezisiert, was das System leisten soll (Abschnitt 4.1 und 4.2), gegen wen es schuetzen soll (Abschnitt 4.3) und welche Schutzziele daraus folgen (Abschnitt 4.4). Es bildet die Grundlage, an der die Architektur (Kapitel 5) und die Evaluation (Kapitel 8) gemessen werden.

4.1 Funktionale Anforderungen

- F1 Tunnel beliebiger Protokolle.** Das System tunnelt bidirektionale TCP-Stroeme sowie UDP-Datagramme zwischen einem Client und einem Origin. Datagrammgrenzen bleiben erhalten.
- F2 Enrollment und Identitaet.** Ein Agent registriert sich ueber ein einmaliges Join-Token bei der Steuerebene und bindet dabei einen ed25519-Schluessel; kurzlebige Anmeldedaten dienen als mTLS-aehnlicher Nachweis gegenueber dem Edge.
- F3 Rendezvous.** Ein Client loest ein opakes Routing-Token beim Edge zum zustaendigen Tunnel auf, ohne einen Hostnamen zu nennen.
- F4 Missbrauchsschranke.** Die Rendezvous-Anbahnung ist durch einen PoW-Nachweis verteuert; die Schwierigkeit ist konfigurierbar.
- F5 Direkter Pfad und Fallbacks.** Bei Erreichbarkeit nutzt der Client einen direkten P2P-Pfad zum Agent; andernfalls den Edge-Relay. Ist UDP blockiert, traegt ein TLS-ueber-TCP-Fallback denselben Tunnel.
- F6 Steuerebene.** Enrollment, Registry/Rendezvous und Abrechnung sind ueber einen laufenden Dienst erreichbar.
- F7 Beobachtbarkeit.** Der Agent exponiert Prometheus-Metriken (Tunnelzahl, Bytes, Handshake-Latenz).

F8 Konten und Zahlung. Ein pseudonymes Prepaid-Guthaben gated die Token-Ausgabe; das Guthaben wird ueber einen Krypto-Payment-Stub aufgeladen.

4.2 Nicht-funktionale Anforderungen

N1 Providerblindheit. Der Betreiber der Vermittlungsebene darf Ziel und Nutzlast strukturell nicht einsehen koennen; er relayt nur Chiffretext.

N2 Kein Vorhalten von Vertrauensmaterial. Die Steuerebene haelt weder Origin-Schluessel noch Nutzlast (thin control plane).

N3 Beschraenkter Overhead. Der Latenz-Overhead gegenueber der reinen Netzlaufzeit soll klein und – ueber Netzbedingungen – vorhersagbar sein (empirisch belegt in der Evaluation).

N4 Reproduzierbarkeit. Aufbau, Betrieb und Messung erfolgen ausschliesslich containerbasiert und sind aus dem Repository reproduzierbar.

N5 Pseudonymitaet. Konten werden ausschliesslich ueber opake Kennungen gefuehrt; es werden keine personenbezogenen Daten gespeichert.

4.3 Bedrohungsmodell

4.3.1 Akteure und Faehigkeiten

A1 Neugieriger Betreiber (honest-but-curious). Der Edge/Betreiber fuehrt das Protokoll korrekt aus, versucht aber, aus dem vermittelten Verkehr Ziel oder Inhalt zu lernen. Er sieht Quell-/Zieladressen der QUIC-Pakete, Zeitpunkte und Volumina, jedoch – so das Ziel – niemals Klartext oder Ziel-Hostnamen.

A2 Aktiver Netz-Angreifer / Zensor. Kann Pakete beobachten, verwerfen und Protokolle grob blockieren (insbesondere UDP). Kann versuchen, sich als Origin auszugeben.

A3 Boesartiger Client / Fluter. Versucht, ueber massenhafte Rendezvous-Versuche Ressourcen zu erschoepfen.

A4 Finanzierter Sybil-Angreifer. Kann beliebig viele pseudonyme Konten eröffnen und bezahlen.

4.3.2 Vertrauensgrenzen und Annahmen

Der Client vertraut der *Origin-Identitaet*, die er ausserhalb des Bandes als Teil einer Capability erhaelt (Pinning, vgl. Abschnitt 2.3). Der Agent ist *Verwalter* des Origin-Noise-Schlüssels; dieser verlaesst den Agent nie. Die Steuerebene wird als verfuegbar, aber nicht als vertraulich angenommen (N2). Die kryptographischen Primitiven (X25519, ChaCha20-Poly1305, BLAKE2s, ed25519, SHA-256) gelten als sicher.

4.4 Schutzziele und Nicht-Ziele

Schutzziele. **S1 Ende-zu-Ende-Vertraulichkeit und -Integritaet** der Nutzlast gegenueber A1 und A2 (durch Noise-Ende-zu-Ende, Abschnitt 2.3). **S2 Ziel-Blindheit** des Betreibers gegenueber A1 (Adressierung ueber opake Tokens, N1). **S3 Origin-Authentizitaet** gegenueber A2 (Pinning schlaegt fehl bei falschem Schluessel). **S4 Missbrauchsdaempfung** gegenueber A3 (asymmetrischer PoW-Aufwand).

Nicht-Ziele. Ausdruecklich *nicht* adressiert werden: *Anonymitaet* im Sinne der Unverkettbarkeit von Client und Ziel gegenueber einem globalen Beobachter (das ist Tors Ziel, vgl. Abschnitt 3.2); *Traffic-Analyse- und Verschleierungs-Resistenz* gegen A2 (Groessen- und Timing-Seitenkanale bleiben sichtbar); *vollstaendiger DoS-Schutz* (PoW daempft, verhindert aber nicht); und die *oekonomische Abwehr von A4*: ein finanzierter Sybil-Angreifer kann beliebig viele Konten kaufen. Das prepaid-basierte Gating verteuert Missbrauch, loest die Sybil-Oekonomie aber nicht; diese Grenze wird offen benannt und im Backlog gefuehrt.

5 Architektur

Dieses Kapitel beschreibt die Struktur des Systems: die Komponenten und ihre Topologie (Abschnitt 5.1), die zentralen Abläufe (5.2), den Rollen-Dispatch auf einem QUIC-Stream (5.3) und die tragenden Entwurfsentscheidungen (5.4). Es setzt die Anforderungen aus Kapitel 4 in eine Aufteilung von Verantwortlichkeiten um.

5.1 Komponenten und Topologie

Das System besteht aus vier Rollen auf dem Datenpfad – Client, Edge, Agent und Origin – sowie einer Steuerebene (Control Plane). Der Client will einen lokalen Dienst erreichen; das Origin ist der Zieldienst; der Agent ist kundenseitig und haelt den Origin-Schlüssel; der Edge ist der providerblinde Vermittler. Abbildung 5.1 zeigt die Topologie.

Die Trennung ist bewusst: der Edge kennt nur opaque Routing-Tokens und relayt Bytes, das Origin terminiert die Noise-Sitzung, und die Steuerebene haelt weder Schlüssel noch Nutzlast (thin control plane, N2).

5.2 Schlusselfluesse

Enrollment und Anmeldung. Der Agent redeemt ein Join-Token bei der Steuerebene und bindet seinen ed25519-Schlüssel; er erhaelt kurzlebige Anmeldedaten, die der Edge zustandslos verifiziert (F2).

Rendezvous mit Proof-of-Work. Oeffnet ein Client den Tunnel, sendet der Edge eine PoW-Challenge; der Client antwortet mit Loesung und Routing-Token. Erst nach gueltiger Loesung loest der Edge das Token zum Agent auf (F3, F4).

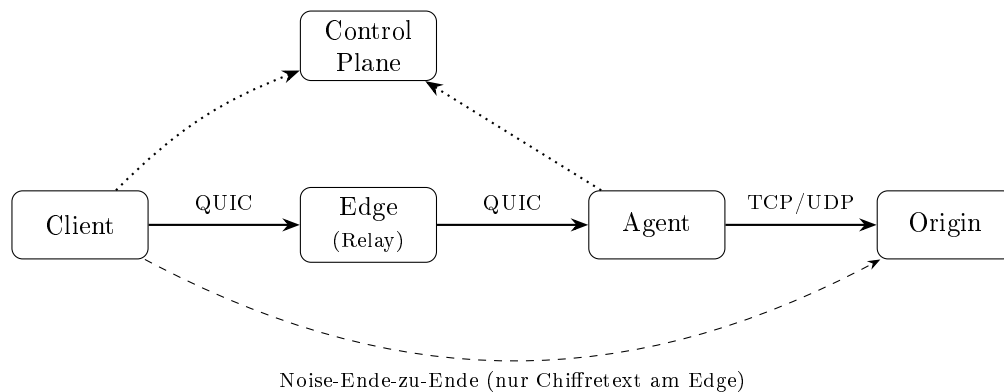


Abbildung 5.1: Topologie: Der Edge relayt zwischen Client und Agent, sieht aber nur den Ende-zu-Ende mit Noise verschluesselten Verkehr (gestrichelter Bogen). Die Steuerebene koordiniert (gepunktet), traegt aber keine Nutzlast.

Relay und Ende-zu-Ende-Handshake. Der Edge verbindet den Client-Stream mit dem registrierten Agent und kopiert fortan nur Bytes. Darueber laeuft der Noise-Handshake zwischen Client und Origin; der Client pinnt die Origin-Identitaet (S1, S3).

Direkter Pfad und Fallbacks. Advertisiert der Agent einen direkten Listener, entdeckt der Client dessen Adresse und Zertifikat ueber den Edge und verbindet direkt; scheitert das, faellt er auf den Relay zurueck. Ist UDP blockiert, traegt ein TLS-ueber-TCP-Kanal denselben Tunnel (F5).

Ablauf eines Tunnelaufbaus. Der vollstaendige Relay-Pfad laeuft in den folgenden Schritten ab:

1. Der Agent registriert sein Routing-Token beim Edge ('A'); der Edge merkt sich Token, Verbindung und die reflexive Adresse des Agents.
2. Der Client oeffnet einen Stream ('C'); der Edge sendet eine PoW-Challenge (Nonce, Schwierigkeit).
3. Der Client loest die Challenge und sendet Loesung und Routing-Token zurueck.
4. Der Edge prueft die Loesung (ein Hash) und loest das Token zum registrierten Agent auf; ist das Token unbekannt oder die Loesung ungueltig, bricht er ab.

Tabelle 5.1: Wire-Format des Rollen-Dispatch (vereinfacht); alle Tokens sind opak, || bezeichnet Konkatenation.

Byte	Anfrage ($\rightarrow$ Edge)	Antwort (Edge)
'A'	Token	OK
'C'	Loesung Token	Route + Byte-Relay
'D'	Token Adresse Zertifikat	OK
'P'	Token	Adresse Zertifikat (oder leer)

5. Der Edge verbindet den Client-Stream mit einem frischen Stream zum Agent und kopiert ab jetzt nur noch Bytes.
6. Ueber diese Bruecke fuehren Client und Origin den Noise-Handshake (Origin-Pinning) und tauschen anschliessend verschluesselte Frames aus; der Edge sieht nur Chiffretext.

Der direkte Pfad kuerzt die Schritte 2–5 ab: der Client fragt den direkten Endpunkt ab ('P'), verbindet sich unmittelbar zum Agent-Listener und fuehrt denselben Noise-Handshake ohne Beteiligung des Edge.

5.3 Rollen-Dispatch auf einem Stream

Client und Agent teilen sich das QUIC-Protokoll zum Edge; die gewuenschte Rolle wird durch ein fuehrendes Byte auf dem Stream ausgewaehlt:

- 'A' Agent registriert einen Tunnel (Token + reflexive Adresse).
- 'C' Client fordert Rendezvous an (PoW), wird geroutet und relayt.
- 'D' Agent advertisiert seinen direkten Listener (Adresse, Zertifikat).
- 'P' Client fragt den direkten Endpunkt eines Tokens ab.

Tabelle 5.1 fasst das Wire-Format der vier Rollen zusammen. Dieser minimale Dispatch haelt den Edge einfach und erweiterbar, ohne ein separates Steuerprotokoll auf eigener Verbindung.

5.4 Entwurfsentscheidungen

Thin Control Plane. Die Steuerebene koordiniert nur (Enrollment, Registry, Abrechnung) und haelt kein Vertrauensmaterial; ein Kompromiss der Steuerebene gefaehrdet damit nicht die Vertraulichkeit der Tunnel.

Capability ausserhalb des Bandes. Die Verbindungserlaubnis (Routing-Token + Origin-Identitaet + Edge-Adresse) wird vom Kunden out-of-band verteilt; Besitz genuegt, um das Origin zu erreichen und zu authentisieren.

Noise_IK mit Origin-Pinning. Die Wahl des IK-Musters verankert die Origin-Authentizitaet im Client und macht den Edge als Man-in-the-Middle wirkungslos.

Transport-agnostischer Datenpfad. Ein gemeinsamer, bidirektionaler Noise-Streaming-Baustein traegt Streaming, Datagramm, direkten Pfad und TCP-Fallback gleichermaßen; die Betriebsarten unterscheiden sich nur an den Raendern.

6 Implementierung

Der Prototyp ist in Rust als Cargo-Workspace aus fuenf Crates umgesetzt. Dieses Kapitel beschreibt die Aufteilung (Abschnitt 6.1), die gemeinsamen Bausteine (6.2), den Rollen-Dispatch am Edge (6.3) sowie den Daten- und Steuerpfad (6.4).

6.1 Workspace-Struktur

Die Schichtung ist azyklisch: alle Crates haengen nur von `ct-common` ab, nie voneinander (Testabhaengigkeiten ausgenommen). So bleibt der providerblinde Edge unabhaengig vom kundenseitigen Agent.

6.2 Gemeinsame Bausteine (`ct-common`)

Der Kern liegt in `ct-common`. Der Proof-of-Work nutzt fuehrende Nullbits eines SHA-256-Hashes; die Pruefung ist ein einzelner Hash (Listing 6.1).

```
pub fn verify(challenge: &Challenge, solution: &[u8]) -> bool {
    let mut h = Sha256::new();
    h.update(&challenge.nonce);
    h.update(solution);
    leading_zero_bits(&h.finalize()) >= challenge.difficulty
}
```

Listing 6.1: Proof-of-Work: Pruefung der Loesung (vereinfacht).

Der zentrale Datenpfad-Baustein ist `noise_pump`: eine transport-agnostische, bidirektionale Bruecke, die Klartext und verschluesselte Frames in beide Richtungen konkurrenzlos umsetzt (Listing 6.2). Die beiden Richtungen laufen als nebenlaeufige Zweige eines `try_join!`; der `TransportState` wird je Frame unter einem kurz gehaltenen Mutex

Tabelle 6.1: Crates des Workspace und ihre Verantwortung.

Crate	Verantwortung
ct-common	Wire-Typen, Noise, PoW, Framing, Metriken
ct-edge	providerblinder Vermittler (Rollen-Dispatch, Relay)
ct-agent	kundenseitig, Origin-Schlüssel, Serve-Pfad
ct-control-plane	Enrollment, Registry/Rendezvous, Abrechnung
ct-client	Tunnel-Aufbau, Betriebsarten, Bench-Harness

gefuehrt, der *nie* ueber einen `await`-Punkt gehalten wird – so bleibt die Chiffren-Sequenz konsistent, ohne die Nebenlaeufigkeit zu blockieren.

```
tokio::try_join!(
    async { // Klartext -> verschluesseln -> Frame an die Chiffren-Seite
        while let n = plain.read(&mut buf).await? {
            if n == 0 { break; }
            let ct = { ts.lock().write_message(&buf[..n]) };
            cipher.write_all(&frame(&ct)).await?;
        }
        Ok(())
    },
    async { // Frame -> entschluesseln -> Klartext an die lokale Seite
        while let Some(fr) = read_frame(&mut cipher).await? {
            let pt = { ts.lock().read_message(&fr) };
            plain.write_all(&pt).await?;
        }
        Ok(())
    },
)?;
```

Listing 6.2: Bidirektionaler Noise-Pump (vereinfacht).

Die Frames tragen ein 2-Byte-Big-Endian-Laengenpraefix; dieselbe Rahmung erhaelt im UDP-Fall die Datagrammgrenzen (eine Noise-Frame = ein Datagramm). Auf diesem Baustein setzen Streaming, Datagramm, direkter Pfad und TCP-Fallback gleichermassen auf – die Betriebsarten unterscheiden sich nur darin, was an den Enden der Bruecke steht.

6.3 Rollen-Dispatch am Edge (ct-edge)

Der Edge liest je Stream ein fuehrendes Rollen-Byte und verzweigt (Listing 6.3). Der 'C'-Zweig fuehrt das PoW-Rendezvous durch und relayt danach nur noch Bytes.

```
match role {
  b'A' => register_agent(conn, &mut recv, state).await?,
  b'C' => rendezvous_route_relay(conn, state, challenge).await?,
  b'D' => advertise_direct(&mut recv, state).await?,
  b'P' => answer_direct_query(&mut recv, &mut send, state).await?,
  _ => return Err("unknown role".into()),
}
```

Listing 6.3: Rollen-Dispatch (vereinfacht).

Weil Registrierung, Relay und Direktpfad-Abfrage denselben Stream-Typ nutzen, bleibt der Edge ohne separates Steuerprotokoll erweiterbar.

6.4 Daten- und Steuerpfad (agent, client, control-plane)

Agentenseitig terminiert `serve_noise_stream` bzw. `serve_noise_udp` die Noise-Sitzung mit dem Origin-Schluessel und bridget zur lokalen TCP- oder UDP-Instanz; der Datenpfad ist mit Tunnel-Metriken instrumentiert (Tunnelzahl, Bytes je Richtung, Handshake-Latenz). Clientseitig waehlt `client_tunnel_auto` den direkten Pfad, wenn ein Endpunkt advertised ist, und faellt sonst auf den Relay zurueck; bei blockiertem UDP traegt `client_tunnel_noise_tcp` denselben Tunnel ueber TLS-ueber-TCP.

Die Steuerebene stellt Enrollment, Registry/Rendezvous und Abrechnung als HTTP-Dienst bereit; ein `ControlPlaneClient` treibt den vollen Ablauf (Konto eroeffnen, aufladen, Token kaufen, aufloesen) gegen den laufenden Dienst. Die Korrektheit des Zusammenspiels ist durch eine frozen Testsuite belegt, die den Pfad bis hin zum verschlueselten Roundtrip durch echte Komponenten ueberprueft – einschliesslich eines E2E-Tests, in dem das *ueber den bezahlten Flow* ausgegebene Routing-Token einen echten Noise-Tunnel etabliert.

6.5 Beobachtbarkeit

Der Datenpfad ist mit einem dependency-freien Metrik-Set instrumentiert: atomare Zaehler fuer geoeffnete/fehlgeschlagene Tunnel, Bytes je Richtung sowie Handshake-Anzahl und -Latenzsumme. Die Bytes werden ueber einen Metered-Adapter gezaehlt, der den Origin-Socket umhuellet und bei jedem erfolgreichen `poll_read/poll_write` den jeweiligen Zaehler erhoehrt, ansonsten aber transparent durchreicht – so bleibt `noise_pump` unveraendert. Ein optionaler HTTP-Endpoint exponiert die Zaehler im Prometheus-Textformat (Listing 6.4); ein Scraper im Testbett belegt, dass sich die Zaehler bei Tunnel-Aktivitaet bewegen.

```
async fn render(State(m): State<Arc<TunnelMetrics>>) -> impl IntoResponse {
    ([ (CONTENT_TYPE, "text/plain; version=0.0.4"), m.render_prometheus() )
}
```

Listing 6.4: Prometheus-Endpoint (vereinfacht).

6.6 Guthaben-gedeckte Token-Ausgabe

Die Abrechnung koppelt die Token-Ausgabe an ein pseudonymes Prepaid-Guthaben. Entscheidend ist die Reihenfolge (Listing 6.5): es wird *zuerst* abgebucht; schlaegt die Abbuchung fehl (zu wenig Guthaben oder unbekanntes Konto), wird kein Token gepraegt und das Guthaben bleibt unveraendert. Ein Null-Guthaben-Konto erhaelt somit kein Token und kann keinen Tunnel aufbauen. Die Aufladung erfolgt ueber einen idempotenten Payment-Intake-Stub, dessen Bestaetigung ein Konto genau einmal gutschreibt.

```
pub fn issue_token_for_payment(ledger, account, price)
-> Result<RoutingToken, LedgerError> {
    ledger.debit(account, price)?;    // erst abbuchen (atomar) ...
    Ok(RoutingToken(random_32()))    // ... dann praegen
}
```

Listing 6.5: Guthaben-gedeckte Ausgabe (vereinfacht).

7 Produktivierung

Der in Kapitel 6 beschriebene Prototyp weist die Machbarkeit des providerblinden Tunnels nach, ist als Testbett aber noch kein betreibbarer Dienst: der Zustand liegt ausschliesslich im Speicher, es gibt keine konventionelle Nutzeridentitaet, und die Zertifikate sind fest verdrahtet. Dieses Kapitel beschreibt die Ueberfuehrung in ein produktionsreifes System entlang sechs Bausteinen – Persistenz (Abschnitt 7.1), Identitaet (7.2), PKI und Transportsicherheit (7.3), Auslieferung (7.4), Haertung (7.5) und Bezahlung (7.6). Die kryptografische Ende-zu-Ende-Vertraulichkeit der Nutzlast (Noise, [8]) bleibt in allen Bausteinen erhalten: die Produktivierung fuegt Identitaet und Abrechnung hinzu, ohne dem Betreiber Zugriff auf den Klartext zu geben.

7.1 Durabler Zustand

Enrollment, Tunnel-Registry und Guthaben-Ledger wurden von In-Memory-Strukturen auf eine eingebettete SQLite-Datenbank umgestellt. Jeder Speicher kapselt seine Tabellen hinter einer synchron gehaltenen Verbindung; mehrstufige Operationen wie die Zahlungsbestaetigung laufen in einer Transaktion, sodass ein Absturz weder ein Guthaben doppelt gutschreibt noch einen Halbzustand hinterlaesst. Die Durabilitaet ist durch Neustart-Tests abgesichert: eine zweite Dienstinstanz auf derselben Datei sieht denselben Zustand, ein bereits eingeloester Join-Token bleibt verbraucht.

7.2 Identitaet und Authentifizierung

Statt pseudonymer Konten treten konventionelle Konten ueber einen OIDC-Anbieter (Keycloak). Der Kontrollknoten verifiziert Bearer-Token per RS256 gegen den oeffentlichen Realm-Schluessel und prueft Aussteller und Ablauf; das Konto wird aus dem Token-Subjekt abgeleitet, sodass ein Aufrufer stets nur auf sein eigenes Konto wirkt. Bewusst

wird damit die frühere Pseudonymitäts-Aussage aufgegeben – der Betreiber kennt die Identität zur Abrechnung. Die inhaltliche Vertraulichkeit bleibt hingegen bestehen: der Datenpfad ist unverändert Ende-zu-Ende verschlüsselt.

7.3 PKI und Transportsicherheit

Der Edge betreibt eine interne Zertifizierungsstelle. Clients vertrauen der CA-Wurzel statt einem angehefteten Blatt-Zertifikat; dadurch kann der Edge sein Zertifikat rotieren, ohne dass ein Client geändert werden muss. Auf der Transportebene gilt *TLS ueberall*: QUIC mit TLS 1.3 ([11, 9]) und ein TLS-ueber-TCP-Rueckfallpfad zum Edge, HTTPS am Ingress vor dem Kontrollknoten. Der einzige im Klartext sprechende Bestandteil – die HTTP-Schnittstelle des Kontrollknotens – verlässt den Cluster nie.

7.4 Auslieferung

Der Dienst ist auf zwei Wegen auslieferbar. Für den Eigenbetrieb genügt ein gehärtetes Compose-Bündel mit persistentem Datenvolumen, Neustart-Richtlinie und einer Bereitschaftsprüfung auf `/readyz`. Für den gehosteten Betrieb existieren Kubernetes-Manifeste (`kustomize`) mit einem PVC-gestützten Kontrollknoten, Liveness-/Readiness-Sonden, gehärteten Sicherheitskontexten und einem TLS-terminierenden Ingress. Beide Wege nutzen dieselben Binaries und dasselbe Protokoll.

7.5 Härtung

Neben dem Proof-of-Work-Gatter ([1]) begrenzt ein Ratenlimit je Konto die Token-Ausgabe, sodass ein einzelnes Konto den Kontrollknoten nicht erschöpfen kann. Die Abhängigkeiten werden gegen die RustSec-Advisory-Datenbank geprüft und über eine eingetragene `Cargo.lock` exakt gepinnt; ein Wächter blockiert versehentlich eingetragene Geheimnisse. Ein aktualisiertes Bedrohungsmodell hält fest, was der Dienst leistet und was nicht – insbesondere macht er keine Anonymitätsaussage.

7.6 Bezahlung

Die Guthaben-Gutschrift wird nur noch durch ein signiertes Ereignis des Zahlungsanbieters ausgelöst, nicht durch einen vom Client aufrufbaren Endpunkt. Der Kontrollknoten verifiziert die HMAC-SHA256-Signatur des Webhooks gegen ein geteiltes Geheimnis und prüft die Frische des Zeitstempels gegen Wiedereinspielung; ohne konfiguriertes Geheimnis wird jeder Webhook abgelehnt (fail-safe). Die Zustellung ist idempotent, sodass ein wiederholtes Ereignis nicht doppelt gutschreibt.

7.7 Zusammenfassung

Mit durablem Zustand, echter Identität, rotierbarer PKI, reproduzierbarer Auslieferung, Härtung und signaturgesicherter Bezahlung ist aus dem Testbett ein betreibbarer Dienst geworden – ohne die zentrale Eigenschaft aufzugeben, dass der Betreiber die Nutzlast nicht lesen kann.

8 Evaluation

Dieses Kapitel beantwortet die Forschungsfragen FF2 (Latenz-Overhead) und FF3 (Netz- und Betriebsartverhalten) empirisch. Es beschreibt das Testbett und die Methodik (Abschnitt 8.1), präsentierte die Ergebnisse (8.2) und ordnet sie samt Limitierungen ein (8.3). Abschnitt 8.4 ergänzt die empirische Leistungsbewertung um eine qualitative Sicherheitsbewertung der Produktivierung aus Kapitel 7.

8.1 Testbett und Methodik

Gemessen wird in der Docker-Compose-Topologie aus Kapitel 5 mit Edge, Agent, Origin und Client auf einem statischen Netz. Link-Stoergroessen werden per `tc netem` auf der Edge-Egress konfiguriert (Verzoegerung, Verlust). Der Client vermisst je Bedingung $n = 20$ vollstaendige Roundtrips mit frisch aufgebauter QUIC-Verbindung; berichtet werden Mittel mit 95%-Konfidenzintervall (CI), Standardabweichung sowie `p50/p95/p99`. Die Matrix umfasst die Betriebsarten One-shot, Streaming und Datagramm $\times$ Verzoegerung $\in \{0, 20, 50\}$ ms $\times$ Verlust $\in \{0, 2\}$ %. Aufbau, Messung und Auswertung sind ausschliesslich containerbasiert und aus dem Repository reproduzierbar (`scripts/sweep.sh`, `plot.sh`, `tabulate.py`).

8.2 Ergebnisse

Tabelle 8.1 fasst alle Bedingungen zusammen. Abbildung 8.1 zeigt die Skalierung mit der Verzoegerung, Abbildung 8.2 den Vergleich der Betriebsarten und Abbildung 8.3 den Einfluss des Verlusts.

Tabelle 8.1: Tunnel-Roundtrip-Latenz je Betriebsart unter emulierten Netzbedingungen (tc netem); Mittel mit 95%-Konfidenzintervall.

Modus	Verzögerung	Verlust	n	Mittel $\pm$ KI (ms)	p50 (ms)	p95 (ms)	p99 (ms)
single	0ms	0%	20	8.8 ± 0.3	8.8	9.7	9.8
single	0ms	2%	20	11.8 ± 3.6	8.7	33.9	36.8
single	20ms	0%	20	131.1 ± 2.0	129.8	133.5	149.6
single	20ms	2%	20	238.9 ± 137.0	129.7	1148.9	1153.7
single	50ms	0%	20	312.4 ± 4.9	309.6	313.6	359.2
single	50ms	2%	20	378.7 ± 103.1	309.7	461.4	1360.8
stream	0ms	0%	20	8.0 ± 0.3	8.0	9.1	9.1
stream	0ms	2%	20	27.5 ± 27.0	9.8	38.7	285.3
stream	20ms	0%	20	131.1 ± 2.1	129.6	134.4	150.6
stream	20ms	2%	20	195.0 ± 99.7	129.8	280.5	1150.0
stream	50ms	0%	20	311.4 ± 5.1	308.6	314.5	360.3
stream	50ms	2%	20	370.8 ± 103.3	309.2	437.5	1363.9
udp	0ms	0%	20	8.3 ± 0.3	8.4	9.2	9.4
udp	0ms	2%	20	9.8 ± 3.1	8.3	9.3	39.6
udp	20ms	0%	20	131.3 ± 2.2	130.0	133.1	151.5
udp	20ms	2%	20	182.3 ± 99.8	130.3	150.0	1149.6
udp	50ms	0%	20	312.6 ± 5.0	309.8	314.2	360.6
udp	50ms	2%	20	450.3 ± 139.8	310.9	1359.1	1359.6

8.3 Diskussion und Limitierungen

FF2 (Latenz-Overhead). Die Baseline (0 ms, 0 %) liegt ueber alle Betriebsarten bei rund 8 ms und misst den Eigenanteil des Tunnels (QUIC-Handshake, PoW-Rendezvous, Mehrsprung-Relay). Mit der Verzoegerung skaliert der Roundtrip linear: fuer single gilt naeherungsweise $RT \approx 8,8 + 6,1 \cdot d$ (mit d = einseitige Edge-Verzoegerung in ms). Der Faktor ≈ 6 spiegelt wider, dass ein Anwendungs-Roundtrip die verzoegerte Strecke mehrfach quert. Bei 0 % Verlust sind die CI eng (± 2 –5 ms), der Zusammenhang also gut aufgeloeset.

FF3 (Netz- und Betriebsartverhalten). Erstens trifft Paketverlust fast ausschliesslich den Tail: bei 2 % Verlust bleibt der p50 praktisch unveraendert, waehrend der p99 explodiert (bei 20 ms von 150 auf 1154 ms, Faktor $\approx 7,7$). Ursache ist der Probe Timeout (PTO) von QUIC – ein waehrend des Handshakes verlorenes Paket kostet ein

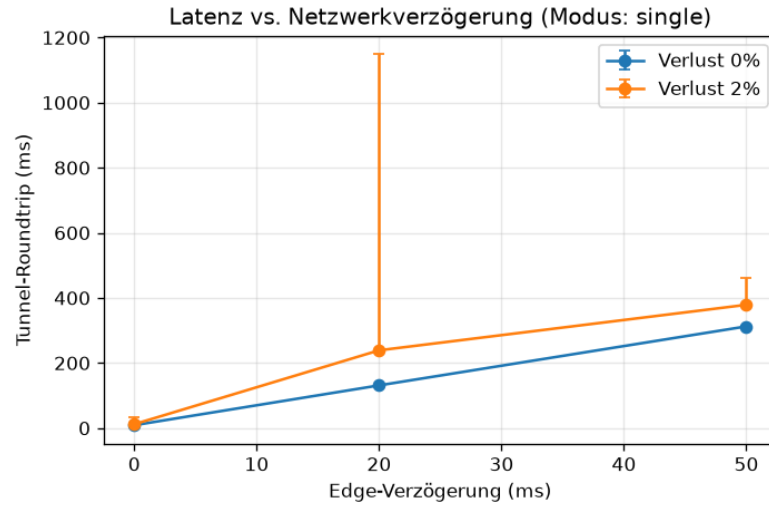


Abbildung 8.1: Tunnel-Roundtrip gegen Edge-Verzögerung (Modus *single*), je eine Kurve pro Verlust; obere Fehlerbalken bis p95.

RTT-proportionales Timeout. Zweitens ist die Betriebsart bei 0 % Verlust nicht latenzbestimmend (Abbildung 8.2): *single*, *stream* und *udp* ueberlagern sich, die Latenz ist verzögerungsdominiert. Unter Verlust streuen die Punktschaetzer zwar, doch die 95%-CI ueberlappen deutlich; ein statistisch belastbarer Modus-Effekt laesst sich aus $n = 20$ nicht ableiten.

Limitierungen. Das Impairment wirkt nur auf der Edge-Egress, nicht symmetrisch – die absolute Latenz ist eine untere Schranke. Der Frisch-Dial pro Iteration ueberzeichnet den Handshake-Anteil bewusst. $n = 20$ loest den Mittelwert bei 0 % Verlust eng auf, ist fuer den schweren Tail unter Verlust aber knapp; die p99-Werte und der Modus-Vergleich unter Verlust sind entsprechend vorsichtig zu lesen. Die PoW-Schwierigkeit ist in diesem Lauf fixiert; die Sweep-Achse erlaubt eine gezielte Schwierigkeitsstudie. Echtes NAT-Hole-Punching bleibt eine Grenze des flachen Container-Testbetts.

8.4 Sicherheitsbewertung der Produktivierung

Die Leistungsmessung erfasst den Eigenanteil des Tunnels, nicht seine Sicherheitseigenschaften. Da ein quantitatives Angriffs-Benchmark den Rahmen dieser Arbeit sprengt,

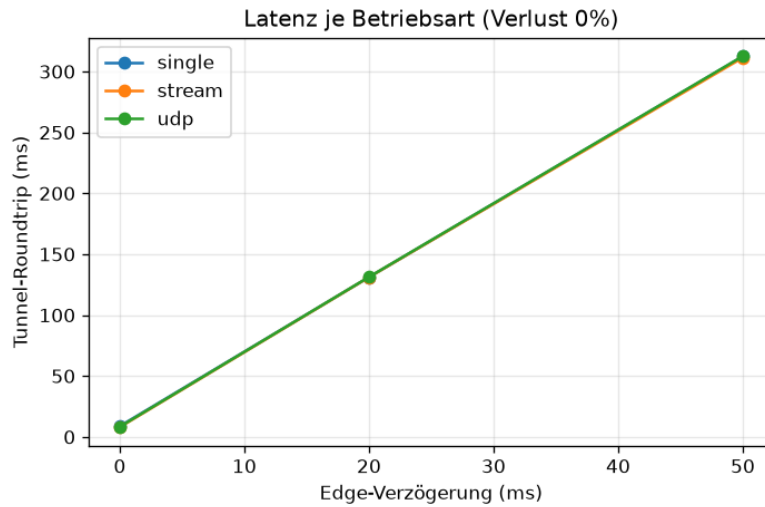


Abbildung 8.2: Vergleich der Betriebsarten (Verlust 0 %): single, stream und udp ueberlagern sich nahezu.

Tabelle 8.2: Angreifer, wirksame Kontrolle und Restrisiko.

Angreifer	Kontrolle	Restrisiko
On-Path-Lauscher	Noise-E2E, TLS 1.3	Nutzlast keins; Metadaten sichtbar
Rotierter/boeser Edge	interne CA, CA-Root-Trust	keins (Ketten-Validierung)
Unauth. Zugriff	OIDC-RS256	an Realm-Schlüssel gebunden
Rendezvous-Flut	PoW + Rate-Limit	begrenzt, nicht ausgeschlossen
Selbst-Gutschrift	signierter Webhook	keins ohne Provider-Secret
Finanzierter Sybil	–	ungeloest

werden die in Kapitel 7 eingefuehrten Kontrollen hier qualitativ gegen ein Angreifermodell bewertet. Tabelle 8.2 ordnet jeder Angreiferklasse die wirksame Kontrolle und das verbleibende Restrisiko zu.

Die zentrale Invariante ist strukturell und nicht politisch: weil der Noise-Handshake ([8]) Client und Origin direkt authentifiziert und verschlüsselt, sieht der Vermittler nur Chiffrat – die Vertraulichkeit der Nutzlast haengt nicht am Wohlverhalten des Betreibers, sondern an der Kryptografie. Ebenso ersetzt die interne CA das Anheften eines Blatt-Zertifikats durch Vertrauen in die CA-Wurzel, sodass eine Zertifikatsrotation die Kette nicht bricht.

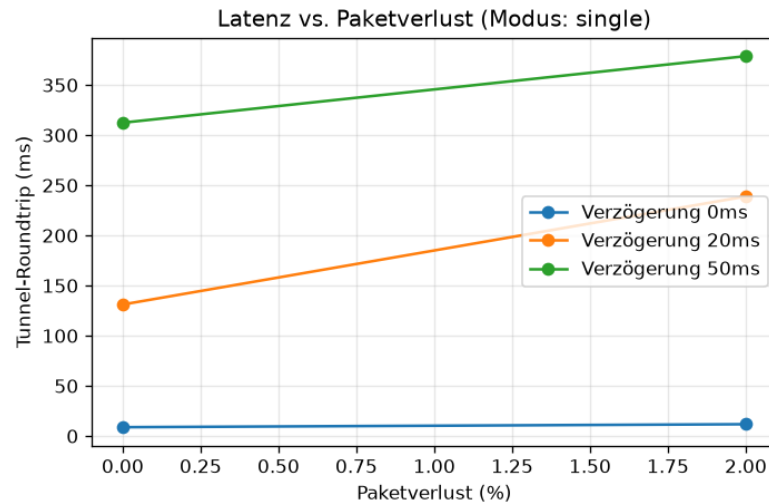


Abbildung 8.3: Tunnel-Roundtrip gegen Paketverlust (Modus *single*), je eine Kurve pro Verzögerung.

Die Verfügbarkeitskontrollen sind bewusst nur graduell: das Proof-of-Work-Gatter ([1]) verteuert Fluten, und das Ratenlimit je Konto verhindert, dass ein einzelnes Konto die Ausgabe erschöpft; einen *finanzierten* Angreifer, der echte Kosten trägt, schrecken beide nicht ab. Dies ist die ehrlich benannte offene Flanke. Auch die Integrität der Abrechnung ist an ein Geheimnis gebunden: ohne konfiguriertes Webhook-Secret wird jede Gutschrift abgelehnt (fail-safe), mit falschem Secret schlägt die HMAC-Verifikation fehl.

Insgesamt sind Vertraulichkeit, Authentifizierung und Abrechnungsintegrität kryptografisch verankert und ohne Zusatzannahmen über den Betreiber wirksam, während Verfügbarkeit gegen finanzierte Gegner und der Schutz von Metadaten bewusst außerhalb des Anspruchs liegen (vgl. das Bedrohungsmodell).

9 Diskussion

Dieses Kapitel bewertet das Ergebnis gegen die Forschungsfragen und das Bedrohungsmodell (Kapitel 4) und benennt die offenen Grenzen.

9.1 Beantwortung der Forschungsfragen

FF1 (Realisierbarkeit). Ein providerblinder Tunnel liess sich konstruieren. Der Edge relayt nachweislich nur Chiffretext: der Ende-zu-Ende mit Noise verschluesselte Verkehr wird byteweise durchgereicht, ohne dass der Vermittler Ziel oder Nutzlast erhaelt. Rendezvous mit Proof-of-Work-Gating, direkter Peer-to-Peer-Pfad mit Relay-Rueckfall und der TLS-ueber-TCP-Fallback funktionieren im Zusammenspiel; die Kette Konto $\rightarrow$ Aufladung $\rightarrow$ guthabengedeckter Token-Ausgabe $\rightarrow$ Tunnel ist ebenfalls durchgaengig belegt. FF1 ist damit fuer Architektur und Kernmechanismen positiv beantwortet.

FF2 (Latenz-Overhead). Der Eigenanteil des Tunnels ist gering (rund 8 ms Baseline) und der Overhead skaliert vorhersagbar linear mit der Netzverzoegerung ($RT \approx 8,8 + 6,1 \cdot d$, siehe Kapitel 8). Der Faktor ≈ 6 ist die wesentliche Kosten: er ruehrt vom mehrfachen Queren der verzoeagerten Strecke durch Handshake, Rendezvous und Relay-Hin-/Rueckweg und liesse sich durch Verbindungs-Wiederverwendung (0-RTT, Keep-Alive) deutlich senken.

FF3 (Netz- und Betriebsartverhalten). Paketverlust trifft den Tail, nicht den Median; die Betriebsart ist bei verlustfreiem Netz nicht latenzbestimmend. Ein statistisch belastbarer Modus-Effekt unter Verlust liess sich bei $n = 20$ nicht nachweisen – ein Hinweis, dass die Transportwahl fuer die reine Latenz zweitrangig ist und primaer nach Semantik (Stream vs. Datagramm) und Zensurresistenz getroffen werden sollte.

9.2 Schutzziele gegen das Bedrohungsmodell

Gegenueber dem neugierigen Betreiber (A1) werden Vertraulichkeit und Ziel-Blindheit (S1, S2) strukturell erreicht: er sieht nur Chiffretext und opake Tokens. Gegenueber dem aktiven Angreifer (A2) verhindert das Origin-Pinning die Man-in-the-Middle-Uebernahme (S3); ein grober UDP-Block wird durch den TCP-Fallback ueberbrueckt. Gegen den Fluter (A3) daempft der asymmetrische Proof-of-Work-Aufwand (S4). Nicht adressiert bleiben – wie im Bedrohungsmodell festgelegt – Traffic-Analyse gegen A2 sowie die Oekonomie des finanzierten Sybil-Angreifers (A4).

9.3 Offene Risiken und Grenzen

Finanzierter Sybil-Angriff (A4). Pseudonyme, prepaid-gedekte Konten verteuern Missbrauch, loesen die Sybil-Oekonomie aber nicht: wer zahlt, kann beliebig viele Tokens erwerben. Eine Abwehr muesste die Kosten pro schaedlicher Aktion ueber den Token-Preis hinaus anheben oder Reputation einfuehren – beides im Spannungsfeld zur Pseudonymitaet.

Traffic-Analyse. Groessen- und Timing-Seitenkanaele bleiben fuer einen beobachtenden Vermittler oder Netz-Angreifer sichtbar. Verschleierung (etwa Padding, konstante Raten) waere eine orthogonale Erweiterung mit eigenem Overhead.

NAT-Hole-Punching. Der direkte Pfad gelingt im flachen Container-Testbett trivial; echtes gleichzeitiges Oeffnen ueber symmetrisches NAT wurde nicht unter realen NAT-Bedingungen erprobt und bleibt eine Testbett-Grenze.

PoW-Parametrisierung. Die Schwierigkeit ist ein Kompromiss: zu niedrig schwaecht die Schranke, zu hoch benachteiligt schwache Clients. Eine adaptive, lastabhaengige Wahl wurde nicht untersucht.

9.4 Methodische Einordnung

Die Messung diente der Charakterisierung des Overheads, nicht einem Vergleich mit Fremdsystemen. Das einseitige Impairment und der Frisch-Dial je Iteration zeichnen ein

konservatives Bild; ein groesseres n und eine breitere Matrix (inkl. Bandbreitenbegrenzung und PoW-Schwierigkeit) wuerden insbesondere die Tail-Aussagen und einen etwaigen Modus-Effekt unter Verlust schaerfen.

10 Fazit und Ausblick

10.1 Zusammenfassung

Diese Arbeit hat einen providerblinden, zensurresistenten Netzwerk-Tunnel entworfen, als Rust-Prototyp aus fuenf Crates implementiert und in einem rein containerbasierten Testbett evaluiert. Der Betreiber der Vermittlungsebene relayt ausschliesslich Chiffretext: die Nutzlast ist mit dem Noise-Protokoll ueber QUIC Ende-zu-Ende verschluesselt und an die gepinnte Origin-Identitaet gebunden, die Adressierung erfolgt ueber opake Tokens statt Hostnamen. Ein Proof-of-Work-Rendezvous daempft Missbrauch, ein direkter Peer-to-Peer-Pfad umgeht bei Erreichbarkeit den Relay, und ein TLS-ueber-TCP-Fallback traegt denselben Tunnel bei blockiertem UDP.

Auf die Forschungsfragen (Kapitel 1) antwortet die Arbeit wie folgt: Ein solcher Tunnel ist realisierbar (FF1) – Architektur und Kernmechanismen wurden bis hin zum verschluesselten Roundtrip durch echte Komponenten belegt, einschliesslich der guthabengedeckten Token-Ausgabe. Der Latenz-Overhead ist gering und vorhersagbar (FF2): der Eigenanteil liegt bei rund 8 ms, und der Roundtrip skaliert linear mit der Netzverzoeigerung ($RT \approx 8,8 + 6,1 \cdot d$). Paketverlust trifft primaer den Tail, und die Betriebsart ist bei verlustfreiem Netz nicht latenzbestimmend (FF3).

Der wesentliche Beitrag ist die *Kombination* von struktureller Providerblindheit, Ende-zu-Ende-Pinning, Proof-of-Work-Rendezvous, direktem Pfad und Transport-Fallback in einem einzigen, lauffaehigen System – ergaenzt um eine reproduzierbare, statistisch charakterisierte Vermessung des dadurch entstehenden Overheads.

Ueber den evaluierten Prototyp hinaus wurde das Testbett zu einem betreibbaren Dienst weitergefuehrt (Kapitel 7): durabler Zustand, konventionelle Identitaet ueber OIDC, eine interne PKI mit Wurzel-Vertrauen, reproduzierbare Auslieferung als Self-Host- und Kubernetes-Variante, Haertung sowie eine signaturgesicherte Abrechnung. Die zentrale

Eigenschaft bleibt dabei erhalten – der Betreiber kann die Nutzlast weiterhin nicht lesen; aufgegeben wurde bewusst nur die Pseudonymität zugunsten konventioneller Konten.

10.2 Ausblick

Aus den in Kapitel 9 benannten Grenzen ergeben sich mehrere Anschlussrichtungen:

- **Traffic-Analyse-Resistenz.** Padding und Ratenglättung könnten Größen- und Timing-Seitenkanäle gegen einen beobachtenden Vermittler verdecken – als orthogonale Schicht mit eigenem, zu vermessenem Overhead.
- **Sybil-Oekonomie.** Der finanzierte Angreifer (A4) bleibt offen: auch die in der Produktivierung eingeführten konventionellen Konten schrecken ihn nicht ab, da er reale Kosten trägt. Kosten pro schädlicher Aktion oder eine kontogebundene Reputation könnten hier ansetzen, wo der reine Token-Preis nicht genügt.
- **NAT-Traversal unter realen Bedingungen.** Ein vollständiges ICE-Verfahren und ein Testbett mit emuliertem symmetrischem NAT würden das Hole-Punching über die flache Bridge hinaus belastbar machen.
- **Overhead-Senkung.** Verbindungs-Wiederverwendung (0-RTT, Keep-Alive) könnte den durch den Frisch-Dial dominierten Handshake-Anteil deutlich reduzieren.
- **Breitere Parameterstudie.** Größeres n , eine Bandbreiten-Achse und eine ausgeführte PoW-Schwierigkeitsachse würden die Tail-Aussagen und einen etwaigen Modus-Effekt unter Verlust schärfen.

Insgesamt zeigt die Arbeit, dass providerblinde Vermittlung nicht als organisatorisches Versprechen, sondern als strukturelle Eigenschaft eines praktisch betreibbaren Tunnels realisierbar ist – zu einem geringen und vorhersagbaren Latenzpreis.

Literaturverzeichnis

- [1] Adam Back. Hashcash – a denial of service counter-measure, 2002. URL <http://www.hashcash.org/papers/hashcash.pdf>.
- [2] Roger Dingledine, Nick Mathewson, and Paul Syverson. Tor: The second-generation onion router. In *13th USENIX Security Symposium*, 2004.
- [3] Jason A. Donenfeld. WireGuard: Next generation kernel network tunnel. In *Network and Distributed System Security Symposium (NDSS)*, 2017.
- [4] David Fifield, Chang Lan, Rod Hynes, Percy Wegmann, and Vern Paxson. Blocking-resistant communication through domain fronting. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2015(2), 2015.
- [5] Bryan Ford, Pyda Srisuresh, and Dan Kegel. Peer-to-peer communication across network address translators. *USENIX Annual Technical Conference*, 2005.
- [6] Jana Iyengar and Martin Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, Internet Engineering Task Force, 2021.
- [7] Ari Keranen, Christer Holmberg, and Jonathan Rosenberg. Interactive connectivity establishment (ICE). RFC 8445, Internet Engineering Task Force, 2018.
- [8] Trevor Perrin. The noise protocol framework. Revision 34, 2018. URL <https://noiseprotocol.org/noise.html>.
- [9] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. RFC 8446, Internet Engineering Task Force, 2018.
- [10] David Schinazi. Proxying UDP in HTTP. RFC 9298, Internet Engineering Task Force, 2022.
- [11] Martin Thomson and Sean Turner. Using TLS to secure QUIC. RFC 9001, Internet Engineering Task Force, 2021.

- [12] Martin Thomson and Christopher A. Wood. Oblivious HTTP. RFC 9458, Internet Engineering Task Force, 2023.
- [13] Eric Wustrow, Scott Wolchok, Ian Goldberg, and J. Alex Halderman. Telex: Anticensorship in the network infrastructure. In *20th USENIX Security Symposium*, 2011.

A Reproduzierbarkeit

Alle Ergebnisse sind aus dem Repository docker-only reproduzierbar; ein Host-seitiges Installieren von Rust, Python oder $\text{\TeX}$ ist nicht noetig.

A.1 Build und Tests

Der Workspace wird hermetisch in einem Rust-Container gebaut und getestet:

```
docker run --rm -v "$PWD":/work -w /work \  
-u $(id -u):$(id -g) -e CARGO_HOME=/tmp/cargo -e HOME=/tmp \  
rust:1-slim sh -c 'cargo build --workspace && cargo test --workspace'
```

Listing A.1: Hermetischer Build und Test des Workspace.

A.2 Testbett-Smokes

Die Topologie und ihre Varianten laufen ueber Docker Compose. Der Basispfad und die Overlays fuer UDP-Origins, direkten Pfad, TCP-Fallback, Metriken und den Control-Plane-Dienst werden jeweils mit `-abort-on-container-exit` gestartet:

```
docker compose -f docker/docker-compose.yml up --build \  
--abort-on-container-exit --exit-code-from client
```

Listing A.2: Basis-Smoke der Topologie.

Die Overlays `docker-compose.udp.yml`, `.p2p.yml`, `.tcp.yml`, `.metrics.yml` und `.controlplane.yml` werden mit einem zusaetzlichen `-f` angehaengt.

A.3 Messung und Auswertung

Die Parameterstudie (Kapitel 8) wird ueber drei Skripte reproduziert:

```
SWEEP_MODES="single stream udp" scripts/sweep.sh    # -> latency.csv
scripts/plot.sh                                     # -> *.png
docker run --rm -v "$PWD":/work -w /work python:3-slim \
  python scripts/tabulate.py                         # -> results-table.{md,
  tex}
```

Listing A.3: Sweep, Abbildungen und Tabellen.

Die Achsen der Matrix (SWEEP_MODES, SWEEP_POWS, SWEEP_DELAYS, SWEEP_LOSSES, SWEEP_ITERATIONS) sind ueber Umgebungsvariablen konfigurierbar.

A.4 Bau dieser Arbeit

Das PDF dieser Arbeit entsteht ueber `scripts/thesis-haw-build.sh` in einem T_EX-Live-Container (`pdflatex` → `bibtex` → `makeglossaries` → `pdflatex` ×2).

Glossar

providerblind Eigenschaft eines Vermittlungsdienstes, den vermittelten Verkehr strukturell nicht einsehen zu koennen: der Betreiber relayt nur Chiffretext und kennt weder Ziel-Hostnamen noch Nutzlast.

Rendezvous Aufloesung eines opaken Routing-Tokens zum aktuell zustaendigen Tunnel, ueber die der Client den passenden Agent erreicht, ohne dem Betreiber einen Hostnamen offenzulegen.

Erklärung zur selbständigen Bearbeitung

Hiermit versichere ich, dass ich die vorliegende Arbeit in allen Teilen selbstständig angefertigt und keine anderen als die in der Arbeit angegebenen Quellen und Hilfsmittel benutzt habe. Die in meiner Arbeit verwendeten KI-basierten Hilfsmittel habe ich (ggf. mit Produktnamen) angegeben.

Ich verantworte die Übernahme jeglicher von mir verwendeter maschinell generierter Passagen vollumfänglich selbst und trage die Verantwortung für eventuell durch die KI generierte fehlerhafte oder verzerrte Inhalte, fehlerhafte Referenzen, Verstöße gegen das Datenschutz- und Urheberrecht oder Plagiate.

Mir ist bewusst, dass wahrheitswidrige Angaben als Täuschungsversuch behandelt werden können.

Ort

Datum

Unterschrift im Original